Touchless interaction controls for Spotify

Project assumptions

The purpose of this project is to create a touchless way to interact with the Spotify Desktop application, so that instead of interacting with Spotify using the keyboard or mouse, the user can perform the predefined hand gestures. The program detects these hand gestures through MediaPipe Hands, and then triggers the corresponding Spotify Desktop keyboard shortcuts through PyAutoGUI.

During development, I changed some of my previously planned gestures, in order to reuse and simplify code, as well as to reduce accidental activations. My final predefined gestures are as follows:

Skip: touch index finger to thumb (ctrl + right arrow)

Go back: touch middle finger to thumb (ctrl + left arrow)

Volume up: touch ring finger to thumb (ctrl + up arrow)

Volume down: touch pinky finger to thumb (ctrl + down arrow)

Pause/play: closed fist (space)

Like/unlike: L shape (alt+shift+B)

Neutral: open palm, fingers spread

The program is designed for Windows, because it relies on the Spotify keyboard shortcuts on Spotify Desktop for Windows – the shortcuts on macOS are different.

System architecture/processing pipeline

The program processes the user input through the following sequence of steps, in order to go from webcam capture to Spotify control.

1. Input capture: Video frames are continuously captured from the webcam using OpenCV
2. Preprocessing: Each frame is mirrored and converted from BGR to RGB
3. Hand tracking: MediaPipe Hands detects the user's hand and returns the positions of the hand's landmarks
4. Gesture recognition: The detected landmarks are passed to my custom functions in gestures.py and recognized by checking the positions and distances between specific fingers
5. Action execution: When a gesture is recognized, the corresponding function in spotify_controls.py is called, after which PyAutoGUI simulates the corresponding keyboard shortcut, which controls the Spotify Desktop application
6. Feedback: The landmarks of the hand are shown with a skeleton overlay, and the last executed action is displayed on the screen

This pipeline is also illustrated in the following diagram[1]

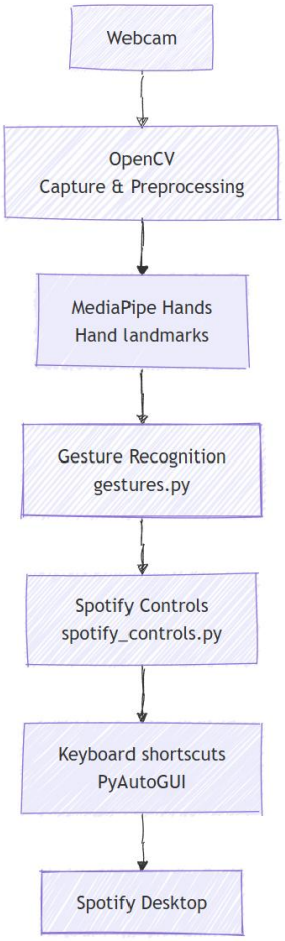


Figure 1: diagram of processing pipeline

[1] Made using https://mermaid.live/

Main algorithms and technical approach

The program is implemented in Python using OpenCV, MediaPipe Hands, and PyAutoGUI.

OpenCV is used to capture the videoframes from the webcam. Each frame is then mirrored for improved user-friendliness, and converted to RGB, and then processed by MediaPipe Hands.

MediaPipe Hands detects the user's hand and returns the position of 21 landmarks of the hand. These landmarks are used in my custom functions in gestures.py to recognize the supported gestures.

Gestures with finger touches are detected by calculating the distance between the thumb tip and the relevant fingertip. If the distance is below the threshold of 0.04, the gesture is activated.

The closed fist and the L-shape gestures are recognized by comparing the positions of the finger tip and the PIP landmarks to determine whether each finger is extended or folded.

When the gesture is recognized, the program then calls the corresponding function in spotify_controls.py. These functions use PyAutoGUI to simulate Spotify keyboard shortcuts, which allows the user to control the playback without using the keyboard or mouse.

To avoid the same gesture triggering repeatedly, I've added a one-second delay after each recognized action, before another action can be executed, using last_action_time and action_delay.

In general, I have repeated the approach I've used in earlier lab reports for MediaPipe exercise, to use something I am familiar with, and that I know works.

User interface design

The user interface design is simple, consisting of the camera preview window created with OpenCV. The purpose of this interface is to give the user visual feedback while using the hand gestures.

The camera window shows the live webcam image. A skeleton of the detected landmarks of the hand are shown as an overlay over the camera preview, so the user can see if the hand is being tracked correctly.

At the top left corner of the window, the last executed action is shown in a bright green text, in order to help the user know which gesture was recognized and if the intended Spotify keyboard command was triggered.

Note that the Spotify Desktop application is still used as the music player interface – this Python program only works as an additional touchless interaction layer on top of Spotify.

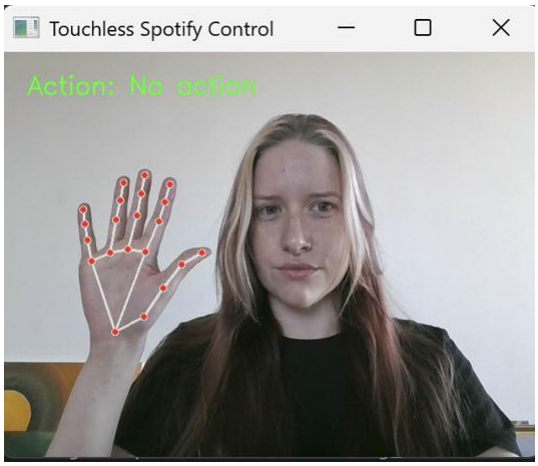


Figure 2: screenshot showing the camera window with the hand skeleton overlay. No action has been triggered yet

Execution instructions

The program is designed to run on Windows with the Spotify Desktop application installed.

The user needs Python 3.12, a webcam, and the Python libraries listed in requirements.txt.

To run the system:

1. Open the Spotify Desktop application
2. Start the program by running main.py
3. Make sure Spotify Desktop is the active window
4. Perform the supported hand gestures
5. To exit the program, press the Q key while the camera window is active

Supported gestures:

- Touch index finger to thumb: skip (next track)
- Touch middle finger to thumb: go back (previous track)
- Touch ring finger to thumb: volume up
- Touch pinky finger to thumb: volume down
- Thumb and index finger out with closed fist (L sign): like/unlike song
- Closed fist: play/pause song

It is recommended to keep an open palm with spread fingers as a neutral/resting position.

Example results

I've tested the program using the Spotify Desktop application on Windows with my laptop's built-in webcam. I did this in both a low-light setting and in bright daylight. Both worked fine

The following gestures were successfully recognized, and triggered the expected actions:

Gesture	Expected action
Thumb touching index finger	Next track
Thumb touching middle finger	Previous track
Thumb touching ring finger	Volume up
Thumb touching pinky finger	Volume down
Closed fist	Play/pause
L-shape	Like/unlike current song

During testing, the landmarks of the hands were correctly displayed, and the last recognized action was shown in the camera window. The one-second delay between actions prevented the same action from accidentally being triggered repeatedly while the gesture was held.

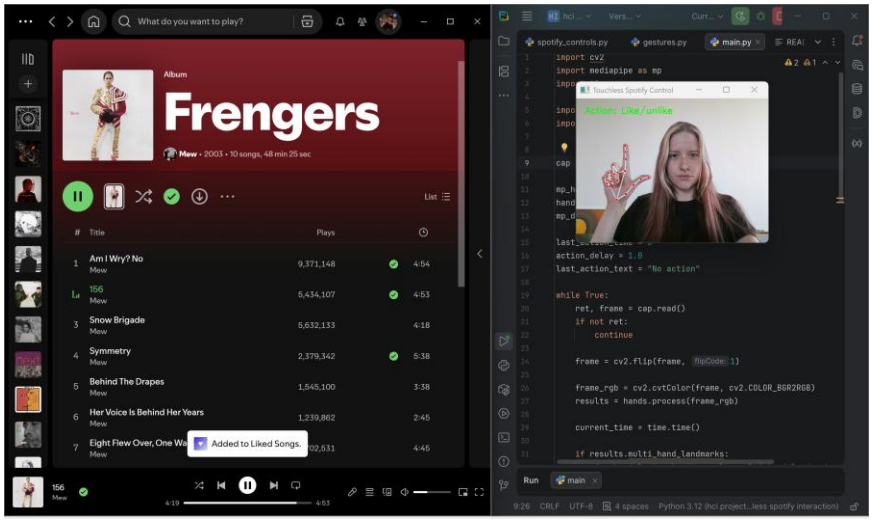


Figure 3: screenshot showing the gesture for liking a song working as expected

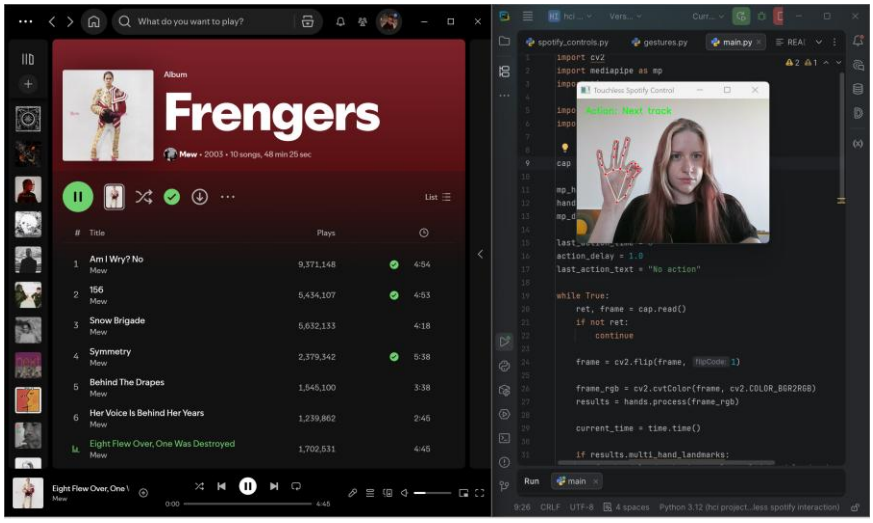


Figure 4: screenshot showing the skip song gesture working as expected